

ERPGuard-AI

Technical Documentation

AI-Powered ERP Compliance & Sales Intelligence System

Version 0.1.0 | Proof of Concept
Python 3.13+ | LangGraph | OpenAI



Md. Golam Mostofa
Sr. AI Engineer
Golammostofa10001@gmail.com

1. System Overview

ERPGuard-AI is an AI-powered chat service designed for ERP environments where compliance, product sales recommendations, vendor onboarding, and operations need to be handled intelligently and securely. The system classifies user intent cheaply, runs deterministic tool chains for each business domain, maintains per-session memory, and emits structured observability traces.

Core design philosophy: All business facts (stock levels, prices, compliance decisions) are owned by deterministic tools that read from a structured data catalog. The LLM is only permitted to **format** the final response — it never decides compliance outcomes, invents product data, or overrides policy rules.

1.1 Key Capabilities

- Sales recommendations with budget filtering and state-based legality checks
- Compliance verification for product availability by US state
- Vendor onboarding checklist validation with document completeness checks
- Operational stock queries with per-warehouse breakdown
- General knowledge base search with user-visibility filtering
- Session memory persistence for multi-turn conversation continuity
- Role-based access control with tool-level permission enforcement
- Structured JSON observability traces for every request

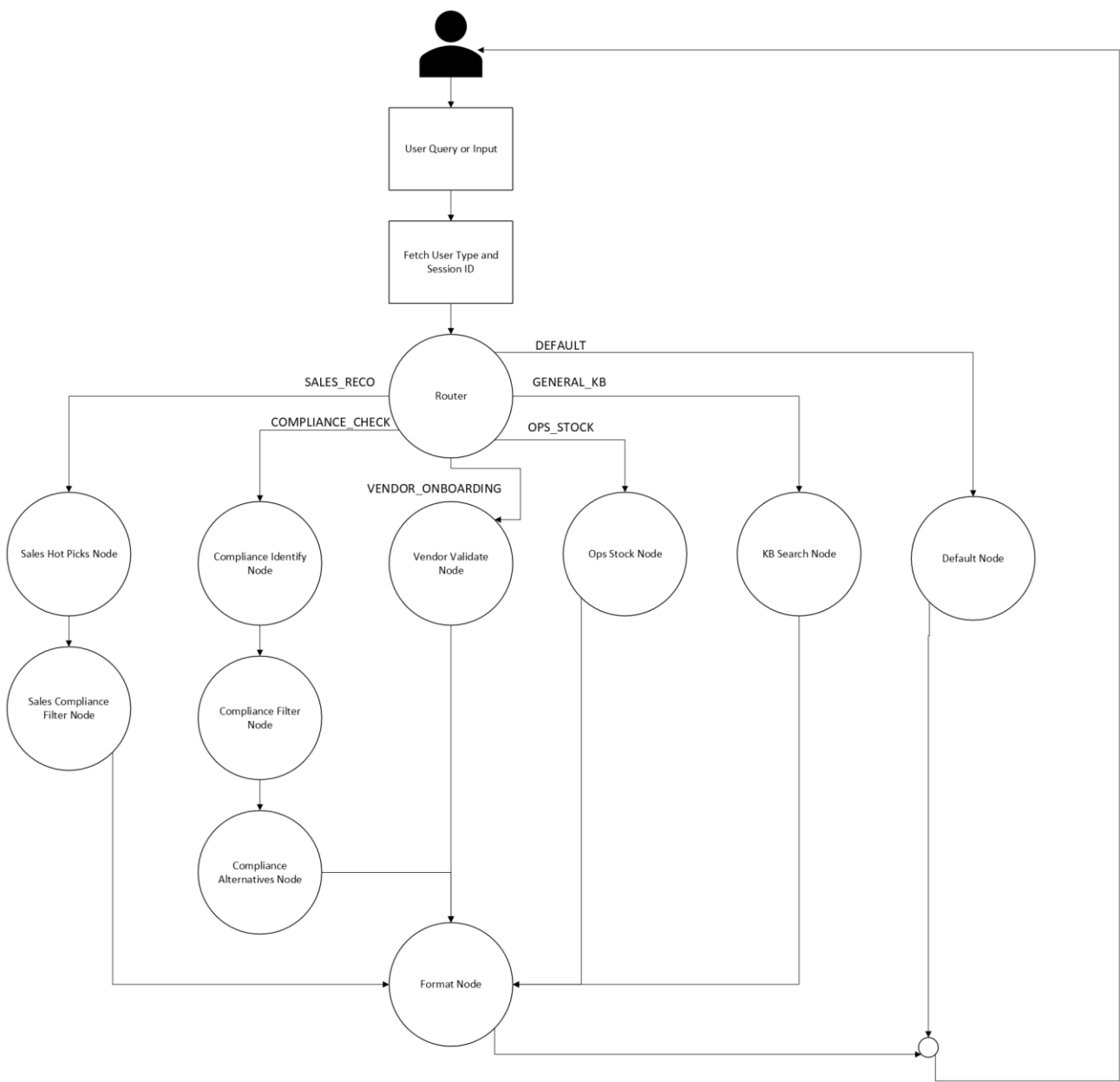
1.2 Technology Stack

Layer	Technology	Purpose
Runtime	Python 3.13+	Core application language
Orchestration	LangGraph 1.1+	DAG-based agentic workflow with checkpointing
LLM	Anthropic Claude / OpenAI GPT	Final response formatting only
Persistence	SQLite (LangGraph checkpointer)	Cross-session state resumption
Package manager	uv	Fast Python dependency management
Data store	data/seed_data.json	Product catalog, KB articles, inventory

2. Architecture

2.1 High-Level Architecture

ERPGuard-AI follows a router-chain pattern. Every user turn enters a single entry point (run_turn), passes through an intent router, is dispatched to a domain-specific processing chain, and exits through a shared format node that generates the human-readable response.

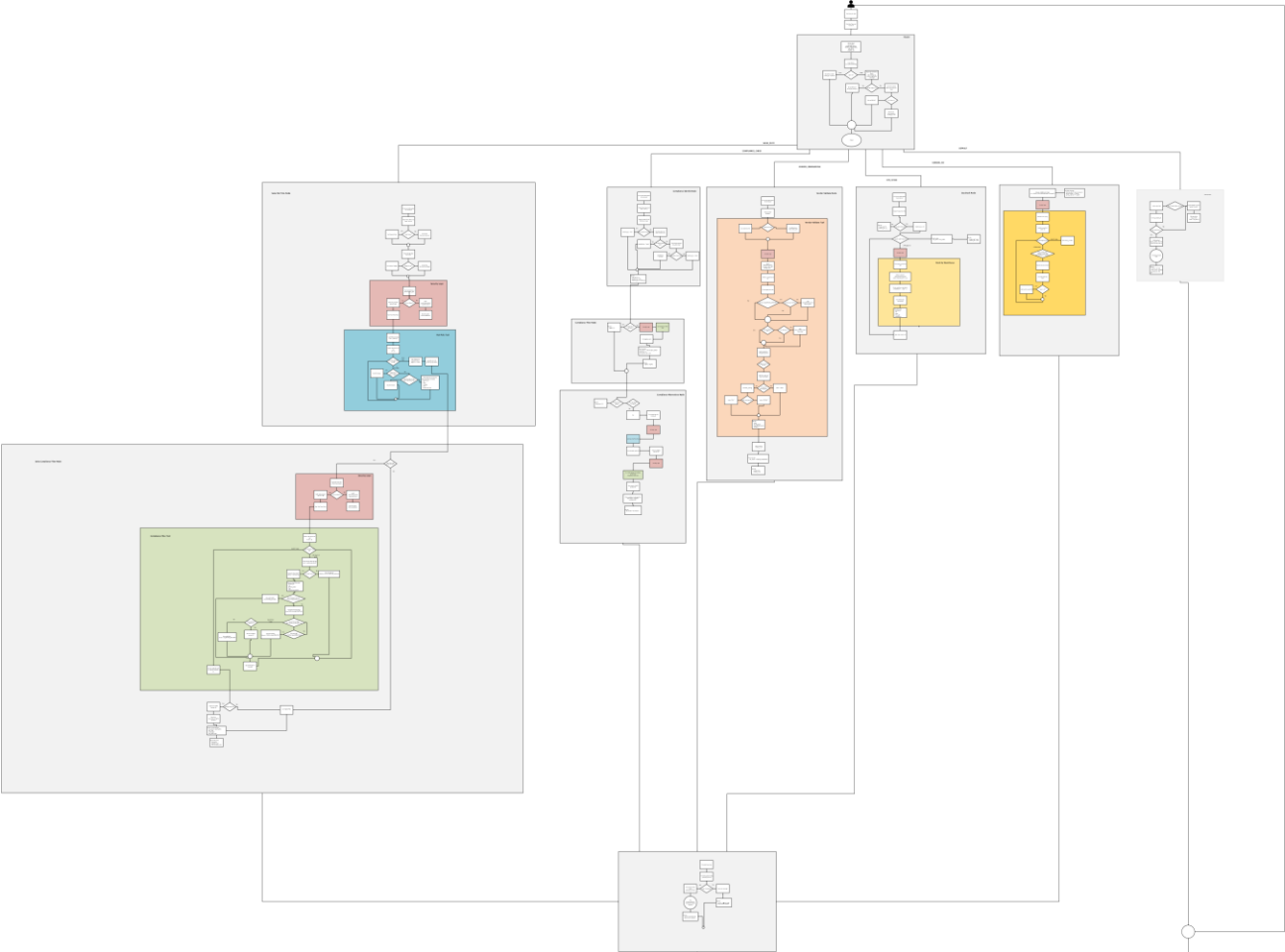


2.2 LangGraph DAG — Node and Edge Map

The system is implemented as a compiled LangGraph StateGraph. Each processing chain is a linear sequence of nodes; all chains converge at the shared format node before termination.

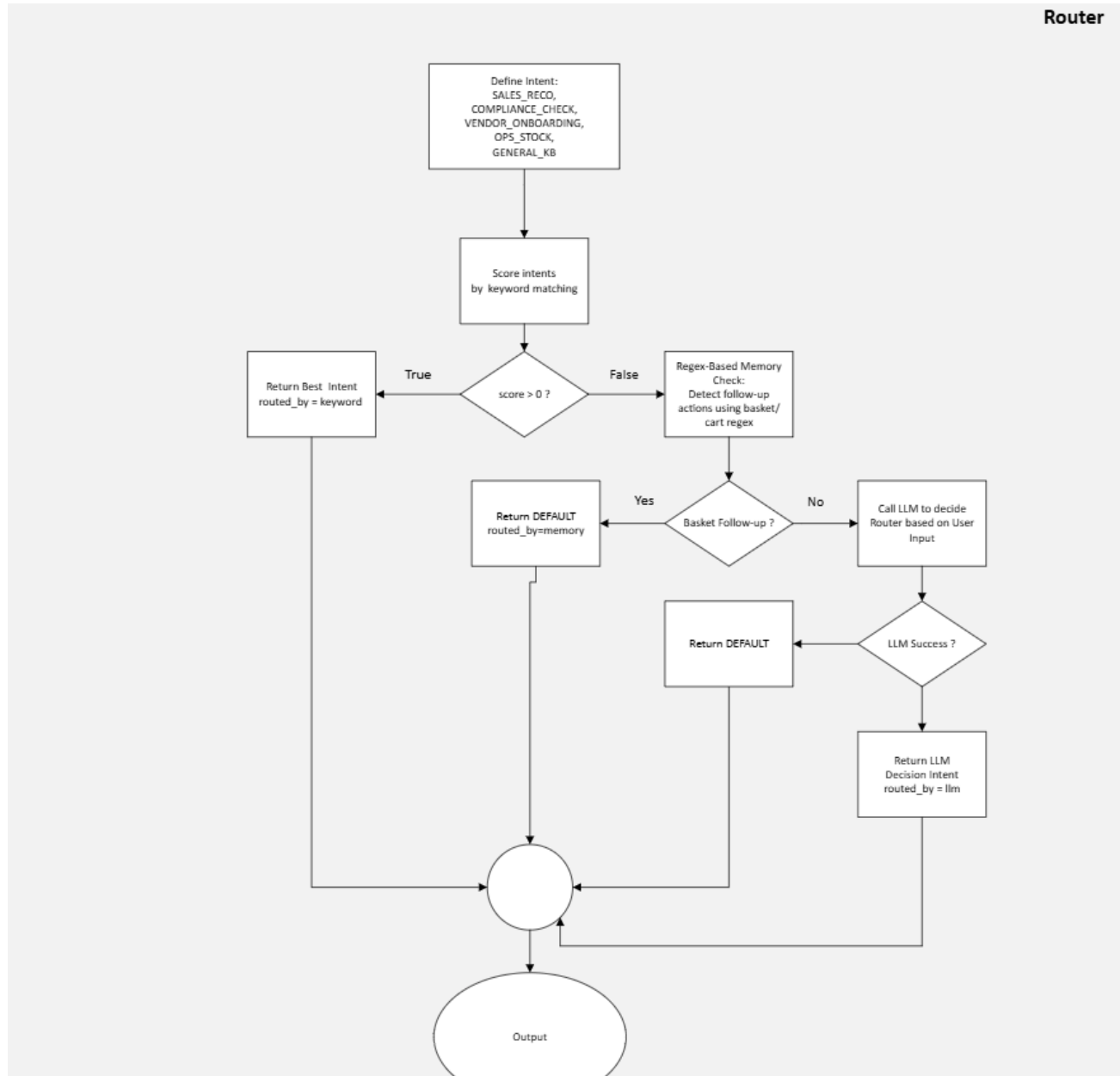
Intent	Node Chain	Tools Invoked
SALES_RECO	sales_hot_picks -> sales_compliance_filter -> format	hot_picks, compliance_filter
COMPLIANCE_CHECK	compliance_identify -> compliance_filter -> compliance_alternatives -> format	compliance_filter, hot_picks
VENDOR_ONBOARDING	vendor_validate -> format	vendor_validate
OPS_STOCK	ops_stock -> format	stock_by_warehouse
GENERAL_KB	kb_search -> format	kb_search
DEFAULT	default -> END (no format node)	None (memory basket follow-up)

2.3 Low-Level Flow



N.B: For best view, you can find the architecture: <https://github.com/shuvo881/ERPGuard-AI/blob/main/docs/low%20level%20architecture.pdf>

2.4 Router Decision Logic



The router applies a three-tier classification strategy designed to minimize LLM calls and cost:

Tier 1 — Keyword Scoring:

The system first scores intents using keyword matching.

Example:

- "recommend products" → SALES_RECO
- "is this legal in Utah?" → COMPLIANCE_CHECK
- "inventory status" → OPS_STOCK

Tier 2 — Memory Follow-up Detection:

The router checks for follow-up shopping behavior using basket/cart regex patterns.

Examples:

- "add this to cart"
- "buy the first one"
- "checkout"

Tier 3 — LLM Structured-Output Fallback:

If no keyword intent is detected, the system calls an LLM router.

The LLM classifies the request into one of the supported intents.

2.5 Session Memory and Checkpointing

ERPGuard-AI maintains two layers of session state:

In-process session store (src/session/store.py): A thread-safe dict keyed by session_id. Holds last_intent, last_state, last_budget, and last_product_ids. Used for fast intra-session slot resolution without re-parsing.

LangGraph checkpointer: Persists the full LangGraph State TypedDict per thread_id = session_id. Default backend is SqliteSaver writing to data/checkpoints.sqlite. Set ERPGUARD_CHECKPOINT_SQLITE=0 to use the in-memory MemorySaver instead.

These two layers together allow follow-up queries like 'add 2 of the first one to the basket' to resolve against product IDs surfaced in the prior turn, even after a process restart when SQLite persistence is enabled.

2.6 Security Layer

Every tool invocation passes through a centralized security layer.

2.6.1 Permission Validation

The system validates: Access Allowed?

based on:

- User Type
- Requested Tool
- Visibility Rules

2.6.1 Audit Logging

Every access attempt is logged.

Successful: Audit: tool_invoked

Denied: Audit: permission_denied

2.7 SALES_RECO Workflow

2.7.1 Objective

Generate compliant product recommendations based on:

- User state
- Budget
- Product restrictions

2.7.2 Product Recommendation Pipeline

Step 1 — Load Product Catalog

```
rows = products()
```

Step 2 — Budget Filtering

Products exceeding the user budget are skipped.

Condition:

```
price > budget
```

Step 3 — State Restriction Filtering

Blocked products are removed.

Condition:

```
state in blocked_states
```

Step 4 — Build Recommendation Output

Allowed products append metadata:

- product_id
- sku
- name

- category
- price
- popularity_score

Step 5 — Sort Recommendations

Products are sorted:

Highest popularity → Lowest popularity

Step 6 — Apply Top-N Limit

Only the top N products are returned.

2.8 Compliance Filter Workflow

2.8.1 Objective

Validate whether products are legally allowed in a given state.

2.8.2 Product Validation Logic

Each product undergoes:

A. Blocked State Validation

```
if state_u in blocked_states:  
    return BLOCKED
```

B. Restricted Flag Validation

The system checks:

STATE_FLAG_RESTRICTIONS

Examples:

- THC
- Kratom
- Mushroom

If restricted:

BLOCKED

C. Lab Report Requirement

If:

```
lab_report_required == True
```

Return:

REVIEW

D. Approved Products

Otherwise:

ALLOWED

2.9 Compliance Alternatives Workflow

If products are blocked:

1. Retrieve alternative products
2. Re-run compliance checks
3. Keep only allowed alternatives
4. Return top 3 alternatives

Output:

```
{  
  "alternatives": [...]  
}
```

2.10 Vendor Onboarding Workflow

2.10.1 Objective

Validate vendor-submitted product attributes.

2.10.2 Validation Process

Step 1 — Extract Vendor Attributes

Attributes are extracted from:

- User input
- Session fallback memory

Step 2 — Required Field Validation

Checks:REQUIRED_FIELDS

Missing fields are collected.

Step 3 — Lab Report Validation

For regulated categories: LAB_REPORT_CATEGORIES

Missing reports trigger: required_documents.append("LAB_REPORT_COA")

Step 4 — State Restriction Validation

Ensures: state_restrictions

exists and is a valid list.

Step 5 — Final Status Determination

Possible statuses:

Status	Meaning
PASS	Fully valid
REVIEW	Requires manual review
FAIL	Critical required fields missing

2.10.3 Vendor Output

```
{
  "status": "...",
  "missing_fields": [...],
  "required_documents": [...],
  "checklist": [...]
}
```

2.11 OPS_STOCK Workflow

2.11.1 Objective

Provide warehouse-level inventory visibility.

2.11.2 Stock Processing Pipeline

Step 1 — Extract Product IDs

If no product IDs are found:

```
{
  "stock_rows": [],
  "product_ids": []
}
```

Step 2 — Fetch Product Inventory

For each product:

- Load inventory records
- Group warehouse quantities
- Calculate total quantity

Step 3 — Warehouse Aggregation

Warehouse objects:

```
{
  "warehouse": "...",
  "qty": ...
}
```

Step 4 — Sort Inventory

Rows sorted by:

Highest quantity → Lowest quantity

2.12 Knowledge Base Workflow (GENERAL_KB)

2.12.1 Visibility Filtering

Documents are filtered by user visibility:

User Type	Visibility
internal_sales	Full access
portal_vendor	Vendor-only
portal_customer	Public-only

2.12.2 Similarity Search

The system:

1. Tokenizes user query
2. Tokenizes document text
3. Computes similarity score

Only documents with score > 0

are returned (top_k).

2.13 Default Conversational Workflow

If no intent matches:

1. Load session data
2. Check basket/cart follow-up
3. Generate fallback conversational response
4. Use LLM formatting layer

Example fallback:

"I couldn't classify your request."

2.14 Response Formatting Engine

2.14.1 Payload Builder

The system builds intent-specific payloads using: `_payload_for(state)`

2.14.2 Formatting Layer

The formatting engine:

1. Loads system prompt templates
2. Maps error messages
3. Calls: `format_with_llm()`

2.14.3 Final Response

Returns:

```
{
  "output": formatted_text,
  "tool_results": payload
}
```


3. Module Reference

3.1 Entry Point — main.py

Provides two execution modes:

- Interactive REPL: prompts for `user_type` and `session_id`, then loops on user input.
- Demo mode (`--demo` flag): replays five scripted prompts through a single session and writes traces to `logs/trace.jsonl`.

3.2 Orchestrator — src/orchestrator/graph.py

Function / Symbol	Description
<code>build_graph()</code>	Compiles and caches the LangGraph StateGraph. Called once per process.
<code>run_turn(...)</code>	Primary entry point. Accepts <code>user_input</code> , <code>user_type</code> , <code>session_id</code> . Creates a <code>TraceEvent</code> , invokes the graph, returns structured result dict.
<code>get_checkpoint(session_id)</code>	Returns the latest persisted State for a session. Useful for debugging.
<code>clear_session(session_id)</code>	Resets graph state for a session. Used in tests and demos.
<code>new_session_id()</code>	Generates an 8-character hex UUID4 session identifier.
<code>_checkpointer()</code>	Selects <code>SqliteSaver</code> or <code>MemorySaver</code> based on <code>ERPGUARD_CHECKPOINT_SQLITE</code> env var. Cached via <code>lru_cache</code> .

3.3 Router — src/router/intent_router.py

Function	Description
<code>keyword_route(text)</code>	Returns (intent, score). Scores each intent by summing keyword hits in lowercased input.
<code>is_basket_followup(text)</code>	Regex-based heuristic for cart/basket follow-up detection.
<code>llm_call_router(state)</code>	Main router node. Applies three-tier logic and returns {decision, routed_by} dict for LangGraph state.
<code>route_decision(state)</code>	LangGraph conditional-edge selector. Returns the intent string from state.

3.3.1 Intent Keywords

Intent	Trigger Keywords
SALES_RECO	hot pick, best seller, recommend, popular, under \$, budget, show me product

COMPLIANCE_CHECK	legal, allowed, blocked, ban, compliance, restricted, why is, alternative, lab report, coa
VENDOR_ONBOARDING	vendor, onboard, upload, checklist, missing field, net wt, net weight, fix
OPS_STOCK	stock, inventory, warehouse, in stock, how much, qty, quantity, available
GENERAL_KB	policy, policies, sop, shipping, return, guide, how do, what is the

3.4 Deterministic Tools — src/tools/

All tools are fully deterministic: they read from seed_data.json and apply static policy tables. The LLM cannot override their outputs.

3.4.1 hot_picks (src/tools/hot_picks.py)

Signature: hot_picks(state, budget, limit=5) -> list[dict]

Returns the top N products by popularity_score that are within budget and not blocked in the given state. Performs a preliminary state filter (blocked_states list) but does not enforce flag-level restrictions — that is delegated to compliance_filter.

3.4.2 compliance_filter (src/tools/compliance.py)

Signature: compliance_filter(state, product_ids) -> list[dict]

Returns ALLOWED, BLOCKED, or REVIEW for each product_id in the given US state. Decision logic is applied in order:

- BLOCKED (STATE_BLOCKED): product is in the blocked_states list for that state.
- BLOCKED (STATE_FLAG_RESTRICTED): product flags (thc, kratom, mushroom, nicotine, etc.) match a restricted flag for the state per the static STATE_FLAG_RESTRICTIONS table.
- REVIEW (LAB_REPORT_REQUIRED): product requires a COA/lab report on file.
- ALLOWED (OK): no restrictions apply.

3.4.3 stock_by_warehouse (src/tools/inventory.py)

Signature: stock_by_warehouse(product_id) -> dict

Returns total quantity and per-warehouse breakdown for a product, sorted by quantity descending.

3.4.4 vendor_validate (src/tools/vendor.py)

Signature: vendor_validate(attributes_json) -> dict

Validates a vendor product submission against REQUIRED_FIELDS. Returns status (PASS / FAIL / REVIEW), missing_fields, required_documents, and a field-by-field checklist. If the product category requires a lab report (COA) and lab_report_url is absent, the required document is flagged.

3.4.5 kb_search (src/tools/kb.py)

Signature: kb_search(query, top_k, visibility) -> list[dict]

Tokenises the query and scores each KB document by similarity. Applies visibility filtering based on user_type: internal_sales sees all documents, portal_vendor sees vendor-visible docs, portal_customer sees only public docs.

3.5 Schemas — src/schemas/model.py

Type	Kind	Key Fields
State	TypedDict	request_id, session_id, user_type, input, decision, routed_by, picks, legality, alternatives, vendor_result, stock_rows, kb_snippets, tool_results, output
Session	TypedDict	session_id, last_intent, last_state, last_budget, last_product_ids
Route	Pydantic	step: Intent — structured LLM output for routing fallback
Intent	Literal type	SALES_RECO COMPLIANCE_CHECK VENDOR_ONBOARDING OPS_STOCK GENERAL_KB DEFAULT
UserType	Literal type	internal_sales portal_vendor portal_customer

4. Security Model

4.1 Role-Based Tool Access Control

Every tool invocation is gated by a pre-call permission check. The `TOOL_ALLOWLIST` in `src/security/policy.py` maps each tool name to the set of `user_types` permitted to call it. If the calling `user_type` is not in the allowlist, a `PermissionDenied` exception is raised before the tool executes.

Tool	internal_sales	portal_vendor	portal_customer
hot_picks	Yes	No	Yes
compliance_filter	Yes	Yes	Yes
stock_by_warehouse	Yes	No	No
vendor_validate	Yes	Yes	No
kb_search	Yes	Yes	Yes

4.2 PII Redaction

Before any data is handed to an LLM for formatting, the `redact_payload()` function in `src/security/policy.py` recursively traverses the payload and replaces known PII patterns:

- Email addresses are replaced with `<EMAIL>`
- Phone numbers (US formats) are replaced with `<PHONE>`
- Credit card-like digit sequences (13-19 digits) are replaced with `<CARD>`

The current implementation is a stub suitable for a PoC. Production deployments should replace this with a dedicated DLP pipeline such as Microsoft Presidio or a similar NER-based system.

4.3 Audit Logging

Every tool invocation emits two audit events via `src/obs/logger.py`:

- `tool_invoked`: logged on successful permission check. Records tool name, `user_type`, `request_id`, and call arguments.
- `permission_denied`: logged when `assert_allowed()` raises `PermissionDenied`. Records the blocked tool and `user_type`.

Audit events are written to `stderr` (configurable via `ERPGUARD_AUDIT_STDERR`) in append-only JSON format. In production, these should be forwarded to a tamper-evident store such as AWS CloudTrail or a hash-chained audit database.

4.4 LLM Boundaries

The following constraints are hard-coded to ensure the LLM cannot influence business-critical decisions:

- Compliance decisions are computed entirely by the static STATE_FLAG_RESTRICTIONS table. The LLM receives only the pre-computed result and formats it into prose.
- The seed_data.json dataset is never dumped wholesale into a prompt. Only the per-intent payload (typically 5-20 lines) is sent to the LLM.
- Tool allowlists are checked before every call. The LLM cannot trigger a tool the user's role is not permitted to use.
- PII is redacted from all payloads before any LLM hand-off.

5. Observability and Tracing

5.1 TraceEvent Structure

Each call to `run_turn()` produces exactly one `TraceEvent`, accumulated in memory and flushed at the end of the turn as a single JSON line. The event is bound to a `ContextVar` (not to `LangGraph State`) so the checkpointer never serialises it.

Field	Type	Description
<code>request_id</code>	string	12-char hex UUID, unique per turn
<code>user_type</code>	string	internal_sales portal_vendor portal_customer
<code>intent</code>	string	Classified intent for this turn
<code>routed_by</code>	string	keyword memory llm
<code>tools_called</code>	list[dict]	Each entry: name, args, latency_ms, result_summary
<code>total_latency_ms</code>	float	Wall-clock time for the full turn
<code>prompt_token_estimate</code>	int	Approximated from prompt text (4 chars/token heuristic)
<code>error</code>	string null	Exception repr if the turn failed, otherwise null

5.2 Trace Output Destinations

- `stderr`: one JSON line per turn, suitable for log aggregators such as CloudWatch or Datadog.
- `logs/trace.jsonl`: append-only JSONL file. Inspect with: `tail -f logs/trace.jsonl | jq`

5.3 Tool Timing

The `time_tool()` context manager in `src/obs/logger.py` wraps every individual tool call. It records the tool name, input arguments, wall-clock latency in milliseconds, and a `result_summary` dict. This enables per-tool performance attribution within a single request trace.

5.4 Monitoring Queries

Objective	jq command
Tail latest events	<code>tail -f logs/trace.jsonl jq</code>
Errors only	<code>cat logs/trace.jsonl jq 'select(.error != null)'</code>
Slow turns (>2s)	<code>jq 'select(.total_latency_ms > 2000)'</code>

Intent distribution	<code>jq -r '.intent' sort uniq -c</code>
LLM router fallback rate	<code>jq 'select(.routed_by == "llm")'</code>

6. Data Model

6.1 seed_data.json — Catalog Structure

All facts are sourced from data/seed_data.json, loaded once at startup via src/data/loader.py with an lru_cache. The file contains three top-level collections:

6.1.1 products

Field	Type	Description
product_id	int	Primary key
sku	string	Stock keeping unit identifier (e.g. SKU-1001)
name	string	Display name
category	string	Product category (used for lab report requirements)
price	float	Unit price in USD
popularity_score	float	Ranking signal used by hot_picks
blocked_states	list[str]	US state codes where the product is outright blocked
flags	dict[str,bool]	Ingredient flags: thc, kratom, mushroom, nicotine, etc.
lab_report_required	bool	If true, a COA must be on file before shipping

6.1.2 inventory

Each record links a product_id to a warehouse identifier and a quantity. The stock_by_warehouse tool aggregates and sorts these records per product.

6.1.3 kb_docs

Knowledge base articles with visibility levels (public / vendor / internal). Each document has an id, title, text, and visibility field. The kb_search tool applies the caller's visibility before scoring.

6.2 Compliance State-Flag Policy Table

The STATE_FLAG_RESTRICTIONS dict in src/tools/compliance.py is the authoritative policy table for flag-level restrictions. It is static for the PoC but should be versioned and sourced from a policy database in production.

State	Restricted Flags
UT	thc, kratom, mushroom
ID	thc, kratom, mushroom

MA	nicotine
WI	thc
NY	mushroom
CA	(none — all flags permitted)

7. Configuration and Deployment

7.1 Environment Variables

Variable	Default	Description
OPENAI_API_KEY	unset	Alternative to Anthropic key. Used by langchain-openai integration.
ERPGUARD_LLM_MODEL	gpt-5	Override the chat model used for formatting and routing fallback.
ERPGUARD_CHECKPOINT_SQLITE	1	Set to 1 to persist graph state to data/checkpoints.sqlite. Set to 0 for in-memory only.
ERPGUARD_AUDIT_STDERR	1	Set to 0 to suppress audit events from stderr output.

7.2 Installation

Prerequisites: Python 3.13+ and the uv package manager.

```
# Install dependencies
uv sync
# Configure environment
cp .env.example .env
# Edit .env and set ANTHROPIC_API_KEY or OPENAI_API_KEY
# Run scripted demo
uv run python main.py --demo
# Run interactive REPL
uv run python main.py
```

7.3 Operation Without an API Key

ERPGuard-AI runs end-to-end without an LLM API key. In this mode:

- All deterministic tools execute normally against seed_data.json.
- The format node uses a fallback JSON formatter instead of calling an LLM.
- The intent router uses keyword matching only (the LLM fallback branch is skipped).

7.4 Directory Structure

Path	Description
main.py	Entry point: REPL and --demo mode
src/orchestrator/	LangGraph wiring, run_turn(), checkpointer selection

src/router/	Intent router, node functions, common helpers
src/tools/	Deterministic tool implementations
src/schemas/	TypedDicts and Pydantic models
src/security/	Tool allowlists, permission checks, PII redaction
src/session/	In-process session memory store
src/obs/	TraceEvent accumulator, audit logger, time_tool()
src/data/	Data loaders and deterministic slot extractors
src/ai/	LLM model selection and availability check
data/seed_data.json	Product catalog, inventory, and KB articles
data/checkpoints.sqlite	LangGraph session state persistence (auto-created)
logs/trace.jsonl	Append-only structured observability log

8. Demo Scenarios

The following five scenarios are included in the --demo scripted run. They exercise each processing chain and validate end-to-end behaviour.

#	User Type	Prompt	Intent	Chain
1	internal_sales	Give me hot picks for CA under \$5000	SALES_RECO	hot_picks -> compliance -> format
2	internal_sales	Why is SKU-1006 not available in UT? Suggest alternatives.	COMPLIANCE_CHECK	identify -> filter -> alternatives -> format
3	internal_sales	How much stock does SKU-1002 have and where?	OPS_STOCK	ops_stock -> format
4	portal_vendor	I'm uploading a product missing Net Wt and no lab report — what do I fix?	VENDOR_ONBOARDING	vendor_validate -> format
5	internal_sales	Ok add 2 of the first one to the basket	DEFAULT	default -> END

8.1 Inspecting Output

- Structured traces: `tail -f logs/trace.jsonl | jq`
- Persisted graph state: `sqlite3 data/checkpoints.sqlite '.tables'`
- Session checkpoint: use `get_checkpoint(session_id)` programmatically

9. Production Upgrade Considerations

ERPGuard-AI is a proof-of-concept. The following table maps each PoC component to its recommended production equivalent:

PoC Component	Production Replacement	Notes
seed_data.json	Relational DB / ERP connector	Real-time inventory, pricing, and catalog data
In-memory session store	Redis / Memcached	Distributed, TTL-based session memory
SqliteSaver checkpointer	PostgresSaver / RedisSaver	LangGraph supports multiple checkpoint backends
PII redaction stub	Microsoft Presidio / DLP pipeline	NER-based scrubbing with configurable entity types
Audit to stderr	AWS CloudTrail / hash-chained DB	Tamper-evident, queryable audit log
STATE_FLAG_RESTRICTIONS dict	Versioned policy database	Audit trail for policy changes, jurisdiction updates
Keyword scoring	Embedding similarity / fine-tuned classifier	Higher recall for paraphrased or ambiguous inputs
Single-process REPL	FastAPI / gRPC service	Horizontally scalable API with auth middleware

9.1 Scalability Notes

- The LangGraph graph is compiled once per process (lru_cache). In a multi-worker deployment each worker compiles its own instance.
- The in-process session store is not shared across workers. Replace with Redis for multi-instance deployments.
- The seed_data.json loader is cached per process. Replace with a connection-pooled DB client for live data.
- The SQLite checkpointer uses check_same_thread=False, which is safe for a single process with multiple threads but not for multi-process deployments.

10 Future Improvements

Potential enhancements:

- Vector database integration
- Semantic search embeddings
- Real-time compliance API integration

- Multi-state legality engine
- Recommendation ranking ML models
- Streaming LLM responses
- Distributed tracing and observability
- Human-in-the-loop compliance review

11 Conclusion

This architecture provides a scalable, compliance-aware, AI-assisted workflow engine capable of handling:

- Product recommendations
- Compliance verification
- Vendor onboarding
- Inventory operations
- Knowledge retrieval